Chapter 1

Introduction

In this introduction we give an overview of mathematical optimization, focusing on the special role of convex optimization. The concepts introduced informally here will be covered in later chapters, with more care and technical detail.

1.1 Mathematical optimization

A *mathematical optimization problem*, or just *optimization problem*, has the form

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq b_i, \quad i = 1, \ldots, m. \end{array} \tag{1.1}$$

Here the vector $x = (x_1, \ldots, x_n)$ is the *optimization variable* of the problem, the function $f_0 : \mathbf{R}^n \to \mathbf{R}$ is the *objective* function, the functions $f_i : \mathbf{R}^n \to \mathbf{R}$, $i = 1, \ldots, m$, are the (inequality) *constraint functions*, and the constants $b_1, \ldots, b_m$ are the limits, or bounds, for the constraints. A vector $x^\star$ is called *optimal*, or a *solution* of the problem (1.1), if it has the smallest objective value among all vectors that satisfy the constraints: for any z with $f_1(z) \leq b_1, \ldots, f_m(z) \leq b_m$, we have $f_0(z) \geq f_0(x^\star)$.

We generally consider families or classes of optimization problems, characterized by particular forms of the objective and constraint functions. As an important example, the optimization problem (1.1) is called a *linear program* if the objective and constraint functions $f_0, \ldots, f_m$ are linear, *i.e.*, satisfy

$$f_i(\alpha x + \beta y) = \alpha f_i(x) + \beta f_i(y) \tag{1.2}$$

for all $x,\ y \in \mathbf{R}^n$ and all $\alpha,\ \beta \in \mathbf{R}$. If the optimization problem is not linear, it is called a *nonlinear program*.

This book is about a class of optimization problems called *convex optimization problems*. A convex optimization problem is one in which the objective and constraint functions are convex, which means they satisfy the inequality

$$f_i(\alpha x + \beta y) \leq \alpha f_i(x) + \beta f_i(y) \tag{1.3}$$

for all $x,\ y \in \mathbf{R}^n$ and all $\alpha,\ \beta \in \mathbf{R}$ with $\alpha + \beta = 1$, $\alpha \geq 0$, $\beta \geq 0$. Comparing (1.3) and (1.2), we see that convexity is more general than linearity: inequality replaces the more restrictive equality, and the inequality must hold only for certain values of α and β. Since any linear program is therefore a convex optimization problem, we can consider convex optimization to be a generalization of linear programming.

1.1.1 Applications

The optimization problem (1.1) is an abstraction of the problem of making the best possible choice of a vector in $\mathbf{R}^n$ from a set of candidate choices. The variable x represents the choice made; the constraints $f_i(x) \leq b_i$ represent firm requirements or specifications that limit the possible choices, and the objective value $f_0(x)$ represents the cost of choosing x. (We can also think of $-f_0(x)$ as representing the value, or utility, of choosing x.) A solution of the optimization problem (1.1) corresponds to a choice that has minimum cost (or maximum utility), among all choices that meet the firm requirements.

In *portfolio optimization*, for example, we seek the best way to invest some capital in a set of n assets. The variable x_i represents the investment in the ith asset, so the vector $x \in \mathbf{R}^n$ describes the overall portfolio allocation across the set of assets. The constraints might represent a limit on the budget (*i.e.*, a limit on the total amount to be invested), the requirement that investments are nonnegative (assuming short positions are not allowed), and a minimum acceptable value of expected return for the whole portfolio. The objective or cost function might be a measure of the overall risk or variance of the portfolio return. In this case, the optimization problem (1.1) corresponds to choosing a portfolio allocation that minimizes risk, among all possible allocations that meet the firm requirements.

Another example is *device sizing* in electronic design, which is the task of choosing the width and length of each device in an electronic circuit. Here the variables represent the widths and lengths of the devices. The constraints represent a variety of engineering requirements, such as limits on the device sizes imposed by the manufacturing process, timing requirements that ensure that the circuit can operate reliably at a specified speed, and a limit on the total area of the circuit. A common objective in a device sizing problem is the total power consumed by the circuit. The optimization problem (1.1) is to find the device sizes that satisfy the design requirements (on manufacturability, timing, and area) and are most power efficient.

In *data fitting*, the task is to find a model, from a family of potential models, that best fits some observed data and prior information. Here the variables are the parameters in the model, and the constraints can represent prior information or required limits on the parameters (such as nonnegativity). The objective function might be a measure of misfit or prediction error between the observed data and the values predicted by the model, or a statistical measure of the unlikeliness or implausibility of the parameter values. The optimization problem (1.1) is to find the model parameter values that are consistent with the prior information, and give the smallest misfit or prediction error with the observed data (or, in a statistical

framework, are most likely).

An amazing variety of practical problems involving decision making (or system design, analysis, and operation) can be cast in the form of a mathematical optimization problem, or some variation such as a multicriterion optimization problem. Indeed, mathematical optimization has become an important tool in many areas. It is widely used in engineering, in electronic design automation, automatic control systems, and optimal design problems arising in civil, chemical, mechanical, and aerospace engineering. Optimization is used for problems arising in network design and operation, finance, supply chain management, scheduling, and many other areas. The list of applications is still steadily expanding.

For most of these applications, mathematical optimization is used as an aid to a human decision maker, system designer, or system operator, who supervises the process, checks the results, and modifies the problem (or the solution approach) when necessary. This human decision maker also carries out any actions suggested by the optimization problem, *e.g.*, buying or selling assets to achieve the optimal portfolio.

A relatively recent phenomenon opens the possibility of many other applications for mathematical optimization. With the proliferation of computers embedded in products, we have seen a rapid growth in *embedded optimization*. In these embedded applications, optimization is used to automatically make real-time choices, and even carry out the associated actions, with no (or little) human intervention or oversight. In some application areas, this blending of traditional automatic control systems and embedded optimization is well under way; in others, it is just starting. Embedded real-time optimization raises some new challenges: in particular, it requires solution methods that are extremely reliable, and solve problems in a predictable amount of time (and memory).

1.1.2 Solving optimization problems

A *solution method* for a class of optimization problems is an algorithm that computes a solution of the problem (to some given accuracy), given a particular problem from the class, *i.e.*, an *instance* of the problem. Since the late 1940s, a large effort has gone into developing algorithms for solving various classes of optimization problems, analyzing their properties, and developing good software implementations. The effectiveness of these algorithms, *i.e.*, our ability to solve the optimization problem (1.1), varies considerably, and depends on factors such as the particular forms of the objective and constraint functions, how many variables and constraints there are, and special structure, such as *sparsity*. (A problem is *sparse* if each constraint function depends on only a small number of the variables).

Even when the objective and constraint functions are smooth (for example, polynomials) the general optimization problem (1.1) is surprisingly difficult to solve. Approaches to the general problem therefore involve some kind of compromise, such as very long computation time, or the possibility of not finding the solution. Some of these methods are discussed in §1.4.

There are, however, some important exceptions to the general rule that most optimization problems are difficult to solve. For a few problem classes we have

effective algorithms that can reliably solve even large problems, with hundreds or thousands of variables and constraints. Two important and well known examples, described in §1.2 below (and in detail in chapter 4), are least-squares problems and linear programs. It is less well known that convex optimization is another exception to the rule: Like least-squares or linear programming, there are very effective algorithms that can reliably and efficiently solve even large convex problems.

1.2 Least-squares and linear programming

In this section we describe two very widely known and used special subclasses of convex optimization: least-squares and linear programming. (A complete technical treatment of these problems will be given in chapter 4.)

1.2.1 Least-squares problems

A *least-squares* problem is an optimization problem with no constraints (*i.e.*, $m = 0$) and an objective which is a sum of squares of terms of the form $a_i^T x - b_i$:

$$\text{minimize} \quad f_0(x) = \|Ax - b\|_2^2 = \textstyle\sum_{i=1}^k (a_i^T x - b_i)^2. \tag{1.4}$$

Here $A \in \mathbf{R}^{k \times n}$ (with $k \geq n$), a_i^T are the rows of A, and the vector $x \in \mathbf{R}^n$ is the optimization variable.

Solving least-squares problems

The solution of a least-squares problem (1.4) can be reduced to solving a set of linear equations,

$$(A^T A)x = A^T b,$$

so we have the analytical solution $x = (A^T A)^{-1} A^T b$. For least-squares problems we have good algorithms (and software implementations) for solving the problem to high accuracy, with very high reliability. The least-squares problem can be solved in a time approximately proportional to $n^2 k$, with a known constant. A current desktop computer can solve a least-squares problem with hundreds of variables, and thousands of terms, in a few seconds; more powerful computers, of course, can solve larger problems, or the same size problems, faster. (Moreover, these solution times will decrease exponentially in the future, according to Moore's law.) Algorithms and software for solving least-squares problems are reliable enough for embedded optimization.

In many cases we can solve even larger least-squares problems, by exploiting some special structure in the coefficient matrix A. Suppose, for example, that the matrix A is *sparse*, which means that it has far fewer than kn nonzero entries. By exploiting sparsity, we can usually solve the least-squares problem much faster than order $n^2 k$. A current desktop computer can solve a sparse least-squares problem

with tens of thousands of variables, and hundreds of thousands of terms, in around a minute (although this depends on the particular sparsity pattern).

For extremely large problems (say, with millions of variables), or for problems with exacting real-time computing requirements, solving a least-squares problem can be a challenge. But in the vast majority of cases, we can say that existing methods are very effective, and extremely reliable. Indeed, we can say that solving least-squares problems (that are not on the boundary of what is currently achievable) is a (mature) *technology*, that can be reliably used by many people who do not know, and do not need to know, the details.

Using least-squares

The least-squares problem is the basis for regression analysis, optimal control, and many parameter estimation and data fitting methods. It has a number of statistical interpretations, *e.g.*, as maximum likelihood estimation of a vector x, given linear measurements corrupted by Gaussian measurement errors.

Recognizing an optimization problem as a least-squares problem is straightforward; we only need to verify that the objective is a quadratic function (and then test whether the associated quadratic form is positive semidefinite). While the basic least-squares problem has a simple fixed form, several standard techniques are used to increase its flexibility in applications.

In *weighted least-squares*, the weighted least-squares cost

$$\sum_{i=1}^{k} w_i(a_i^T x - b_i)^2,$$

where $w_1, \ldots, w_k$ are positive, is minimized. (This problem is readily cast and solved as a standard least-squares problem.) Here the weights w_i are chosen to reflect differing levels of concern about the sizes of the terms $a_i^T x - b_i$, or simply to influence the solution. In a statistical setting, weighted least-squares arises in estimation of a vector x, given linear measurements corrupted by errors with unequal variances.

Another technique in least-squares is *regularization*, in which extra terms are added to the cost function. In the simplest case, a positive multiple of the sum of squares of the variables is added to the cost function:

$$\sum_{i=1}^{k}(a_i^T x - b_i)^2 + \rho \sum_{i=1}^{n} x_i^2,$$

where $\rho > 0$. (This problem too can be formulated as a standard least-squares problem.) The extra terms penalize large values of x, and result in a sensible solution in cases when minimizing the first sum only does not. The parameter ρ is chosen by the user to give the right trade-off between making the original objective function $\sum_{i=1}^{k}(a_i^T x - b_i)^2$ small, while keeping $\sum_{i=1}^{n} x_i^2$ not too big. Regularization comes up in statistical estimation when the vector x to be estimated is given a prior distribution.

Weighted least-squares and regularization are covered in chapter 6; their statistical interpretations are given in chapter 7.

1.2.2 Linear programming

Another important class of optimization problems is *linear programming*, in which the objective and all constraint functions are linear:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & a_i^T x \leq b_i, \quad i = 1, \ldots, m. \end{array} \tag{1.5}$$

Here the vectors $c, a_1, \ldots, a_m \in \mathbf{R}^n$ and scalars $b_1, \ldots, b_m \in \mathbf{R}$ are problem parameters that specify the objective and constraint functions.

Solving linear programs

There is no simple analytical formula for the solution of a linear program (as there is for a least-squares problem), but there are a variety of very effective methods for solving them, including Dantzig's simplex method, and the more recent interior-point methods described later in this book. While we cannot give the exact number of arithmetic operations required to solve a linear program (as we can for least-squares), we can establish rigorous bounds on the number of operations required to solve a linear program, to a given accuracy, using an interior-point method. The complexity in practice is order $n^2 m$ (assuming $m \geq n$) but with a constant that is less well characterized than for least-squares. These algorithms are quite reliable, although perhaps not quite as reliable as methods for least-squares. We can easily solve problems with hundreds of variables and thousands of constraints on a small desktop computer, in a matter of seconds. If the problem is sparse, or has some other exploitable structure, we can often solve problems with tens or hundreds of thousands of variables and constraints.

As with least-squares problems, it is still a challenge to solve extremely large linear programs, or to solve linear programs with exacting real-time computing requirements. But, like least-squares, we can say that solving (most) linear programs is a mature technology. Linear programming solvers can be (and are) embedded in many tools and applications.

Using linear programming

Some applications lead directly to linear programs in the form (1.5), or one of several other standard forms. In many other cases the original optimization problem does not have a standard linear program form, but can be transformed to an equivalent linear program (and then, of course, solved) using techniques covered in detail in chapter 4.

As a simple example, consider the *Chebyshev approximation problem*:

$$\text{minimize} \quad \max_{i=1,\ldots,k} |a_i^T x - b_i|. \tag{1.6}$$

Here $x \in \mathbf{R}^n$ is the variable, and $a_1, \ldots, a_k \in \mathbf{R}^n$, $b_1, \ldots, b_k \in \mathbf{R}$ are parameters that specify the problem instance. Note the resemblance to the least-squares problem (1.4). For both problems, the objective is a measure of the size of the terms $a_i^T x - b_i$. In least-squares, we use the sum of squares of the terms as objective, whereas in Chebyshev approximation, we use the maximum of the absolute values.

One other important distinction is that the objective function in the Chebyshev approximation problem (1.6) is not differentiable; the objective in the least-squares problem (1.4) is quadratic, and therefore differentiable.

The Chebyshev approximation problem (1.6) can be solved by solving the linear program

$$\begin{array}{ll} \text{minimize} & t \\ \text{subject to} & a_i^T x - t \leq b_i, \quad i = 1, \ldots, k \\ & -a_i^T x - t \leq -b_i, \quad i = 1, \ldots, k, \end{array} \tag{1.7}$$

with variables $x \in \mathbf{R}^n$ and $t \in \mathbf{R}$. (The details will be given in chapter 6.) Since linear programs are readily solved, the Chebyshev approximation problem is therefore readily solved.

Anyone with a working knowledge of linear programming would recognize the Chebyshev approximation problem (1.6) as one that can be reduced to a linear program. For those without this background, though, it might not be obvious that the Chebyshev approximation problem (1.6), with its nondifferentiable objective, can be formulated and solved as a linear program.

While recognizing problems that can be reduced to linear programs is more involved than recognizing a least-squares problem, it is a skill that is readily acquired, since only a few standard tricks are used. The task can even be partially automated; some software systems for specifying and solving optimization problems can automatically recognize (some) problems that can be reformulated as linear programs.

1.3 Convex optimization

A convex optimization problem is one of the form

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq b_i, \quad i = 1, \ldots, m, \end{array} \tag{1.8}$$

where the functions $f_0, \ldots, f_m : \mathbf{R}^n \to \mathbf{R}$ are convex, *i.e.*, satisfy

$$f_i(\alpha x + \beta y) \leq \alpha f_i(x) + \beta f_i(y)$$

for all $x,\ y \in \mathbf{R}^n$ and all $\alpha,\ \beta \in \mathbf{R}$ with $\alpha + \beta = 1$, $\alpha \geq 0$, $\beta \geq 0$. The least-squares problem (1.4) and linear programming problem (1.5) are both special cases of the general convex optimization problem (1.8).

1.3.1 Solving convex optimization problems

There is in general no analytical formula for the solution of convex optimization problems, but (as with linear programming problems) there are very effective methods for solving them. Interior-point methods work very well in practice, and in some cases can be proved to solve the problem to a specified accuracy with a number of

operations that does not exceed a polynomial of the problem dimensions. (This is covered in chapter 11.)

We will see that interior-point methods can solve the problem (1.8) in a number of steps or iterations that is almost always in the range between 10 and 100. Ignoring any structure in the problem (such as sparsity), each step requires on the order of

$$\max\{n^3, n^2m, F\}$$

operations, where F is the cost of evaluating the first and second derivatives of the objective and constraint functions $f_0, \ldots, f_m$.

Like methods for solving linear programs, these interior-point methods are quite reliable. We can easily solve problems with hundreds of variables and thousands of constraints on a current desktop computer, in at most a few tens of seconds. By exploiting problem structure (such as sparsity), we can solve far larger problems, with many thousands of variables and constraints.

We cannot yet claim that solving general convex optimization problems is a mature technology, like solving least-squares or linear programming problems. Research on interior-point methods for general nonlinear convex optimization is still a very active research area, and no consensus has emerged yet as to what the best method or methods are. But it is reasonable to expect that solving general convex optimization problems will become a technology within a few years. And for some subclasses of convex optimization problems, for example second-order cone programming or geometric programming (studied in detail in chapter 4), it is fair to say that interior-point methods are approaching a technology.

1.3.2 Using convex optimization

Using convex optimization is, at least conceptually, very much like using least-squares or linear programming. If we can formulate a problem as a convex optimization problem, then we can solve it efficiently, just as we can solve a least-squares problem efficiently. With only a bit of exaggeration, we can say that, if you formulate a practical problem as a convex optimization problem, then you have solved the original problem.

There are also some important differences. Recognizing a least-squares problem is straightforward, but recognizing a convex function can be difficult. In addition, there are many more tricks for transforming convex problems than for transforming linear programs. Recognizing convex optimization problems, or those that can be transformed to convex optimization problems, can therefore be challenging. The main goal of this book is to give the reader the background needed to do this. Once the skill of recognizing or formulating convex optimization problems is developed, you will find that surprisingly many problems can be solved via convex optimization.

The challenge, and art, in using convex optimization is in recognizing and formulating the problem. Once this formulation is done, solving the problem is, like least-squares or linear programming, (almost) technology.

1.4 Nonlinear optimization

Nonlinear optimization (or nonlinear programming) is the term used to describe an optimization problem when the objective or constraint functions are not linear, but not known to be convex. Sadly, there are no effective methods for solving the general nonlinear programming problem (1.1). Even simple looking problems with as few as ten variables can be extremely challenging, while problems with a few hundreds of variables can be intractable. Methods for the general nonlinear programming problem therefore take several different approaches, each of which involves some compromise.

1.4.1 Local optimization

In *local optimization*, the compromise is to give up seeking the optimal x, which minimizes the objective over all feasible points. Instead we seek a point that is only locally optimal, which means that it minimizes the objective function among feasible points that are near it, but is not guaranteed to have a lower objective value than all other feasible points. A large fraction of the research on general nonlinear programming has focused on methods for local optimization, which as a consequence are well developed.

Local optimization methods can be fast, can handle large-scale problems, and are widely applicable, since they only require differentiability of the objective and constraint functions. As a result, local optimization methods are widely used in applications where there is value in finding a good point, if not the very best. In an engineering design application, for example, local optimization can be used to improve the performance of a design originally obtained by manual, or other, design methods.

There are several disadvantages of local optimization methods, beyond (possibly) not finding the true, globally optimal solution. The methods require an initial guess for the optimization variable. This initial guess or starting point is critical, and can greatly affect the objective value of the local solution obtained. Little information is provided about how far from (globally) optimal the local solution is. Local optimization methods are often sensitive to algorithm parameter values, which may need to be adjusted for a particular problem, or family of problems.

Using a local optimization method is trickier than solving a least-squares problem, linear program, or convex optimization problem. It involves experimenting with the choice of algorithm, adjusting algorithm parameters, and finding a good enough initial guess (when one instance is to be solved) or a method for producing a good enough initial guess (when a family of problems is to be solved). Roughly speaking, local optimization methods are more art than technology. Local optimization is a well developed art, and often very effective, but it is nevertheless an art. In contrast, there is little art involved in solving a least-squares problem or a linear program (except, of course, those on the boundary of what is currently possible).

An interesting comparison can be made between local optimization methods for nonlinear programming, and convex optimization. Since differentiability of the ob-

jective and constraint functions is the only requirement for most local optimization methods, formulating a practical problem as a nonlinear optimization problem is relatively straightforward. The art in local optimization is in solving the problem (in the weakened sense of finding a locally optimal point), once it is formulated. In convex optimization these are reversed: The art and challenge is in problem formulation; once a problem is formulated as a convex optimization problem, it is relatively straightforward to solve it.

1.4.2 Global optimization

In *global optimization*, the true global solution of the optimization problem (1.1) is found; the compromise is efficiency. The worst-case complexity of global optimization methods grows exponentially with the problem sizes n and m; the hope is that in practice, for the particular problem instances encountered, the method is far faster. While this favorable situation does occur, it is not typical. Even small problems, with a few tens of variables, can take a very long time (*e.g.*, hours or days) to solve.

Global optimization is used for problems with a small number of variables, where computing time is not critical, and the value of finding the true global solution is very high. One example from engineering design is *worst-case analysis* or *verification* of a high value or safety-critical system. Here the variables represent uncertain parameters, that can vary during manufacturing, or with the environment or operating condition. The objective function is a utility function, *i.e.*, one for which smaller values are worse than larger values, and the constraints represent prior knowledge about the possible parameter values. The optimization problem (1.1) is the problem of finding the *worst-case* values of the parameters. If the worst-case value is acceptable, we can certify the system as safe or reliable (with respect to the parameter variations).

A local optimization method can rapidly find a set of parameter values that is bad, but not guaranteed to be the absolute worst possible. If a local optimization method finds parameter values that yield unacceptable performance, it has succeeded in determining that the system is not reliable. But a local optimization method cannot certify the system as reliable; it can only fail to find bad parameter values. A global optimization method, in contrast, will find the absolute worst values of the parameters, and if the associated performance is acceptable, can certify the system as safe. The cost is computation time, which can be very large, even for a relatively small number of parameters. But it may be worth it in cases where the value of certifying the performance is high, or the cost of being wrong about the reliability or safety is high.

1.4.3 Role of convex optimization in nonconvex problems

In this book we focus primarily on convex optimization problems, and applications that can be reduced to convex optimization problems. But convex optimization also plays an important role in problems that are *not* convex.

Initialization for local optimization

One obvious use is to combine convex optimization with a local optimization method. Starting with a nonconvex problem, we first find an approximate, but convex, formulation of the problem. By solving this approximate problem, which can be done easily and without an initial guess, we obtain the exact solution to the approximate convex problem. This point is then used as the starting point for a local optimization method, applied to the original nonconvex problem.

Convex heuristics for nonconvex optimization

Convex optimization is the basis for several heuristics for solving nonconvex problems. One interesting example we will see is the problem of finding a *sparse* vector x (*i.e.*, one with few nonzero entries) that satisfies some constraints. While this is a difficult combinatorial problem, there are some simple heuristics, based on convex optimization, that often find fairly sparse solutions. (These are described in chapter 6.)

Another broad example is given by *randomized algorithms*, in which an approximate solution to a nonconvex problem is found by drawing some number of candidates from a probability distribution, and taking the best one found as the approximate solution. Now suppose the family of distributions from which we will draw the candidates is parametrized, *e.g.*, by its mean and covariance. We can then pose the question, which of these distributions gives us the smallest expected value of the objective? It turns out that this problem is sometimes a convex problem, and therefore efficiently solved. (See, *e.g.*, exercise 11.23.)

Bounds for global optimization

Many methods for global optimization require a cheaply computable lower bound on the optimal value of the nonconvex problem. Two standard methods for doing this are based on convex optimization. In *relaxation*, each nonconvex constraint is replaced with a looser, but convex, constraint. In *Lagrangian relaxation*, the Lagrangian dual problem (described in chapter 5) is solved. This problem is convex, and provides a lower bound on the optimal value of the nonconvex problem.

1.5 Outline

The book is divided into three main parts, titled *Theory*, *Applications*, and *Algorithms*.

1.5.1 Part I: Theory

In part I, *Theory*, we cover basic definitions, concepts, and results from convex analysis and convex optimization. We make no attempt to be encyclopedic, and skew our selection of topics toward those that we think are useful in recognizing

and formulating convex optimization problems. This is classical material, almost all of which can be found in other texts on convex analysis and optimization. We make no attempt to give the most general form of the results; for that the reader can refer to any of the standard texts on convex analysis.

Chapters 2 and 3 cover convex sets and convex functions, respectively. We give some common examples of convex sets and functions, as well as a number of convex calculus rules, *i.e.*, operations on sets and functions that preserve convexity. Combining the basic examples with the convex calculus rules allows us to form (or perhaps more importantly, recognize) some fairly complicated convex sets and functions.

In chapter 4, *Convex optimization problems*, we give a careful treatment of optimization problems, and describe a number of transformations that can be used to reformulate problems. We also introduce some common subclasses of convex optimization, such as linear programming and geometric programming, and the more recently developed second-order cone programming and semidefinite programming.

Chapter 5 covers Lagrangian duality, which plays a central role in convex optimization. Here we give the classical Karush-Kuhn-Tucker conditions for optimality, and a local and global sensitivity analysis for convex optimization problems.

1.5.2 Part II: Applications

In part II, *Applications*, we describe a variety of applications of convex optimization, in areas like probability and statistics, computational geometry, and data fitting. We have described these applications in a way that is accessible, we hope, to a broad audience. To keep each application short, we consider only simple cases, sometimes adding comments about possible extensions. We are sure that our treatment of some of the applications will cause experts to cringe, and we apologize to them in advance. But our goal is to convey the flavor of the application, quickly and to a broad audience, and not to give an elegant, theoretically sound, or complete treatment. Our own backgrounds are in electrical engineering, in areas like control systems, signal processing, and circuit analysis and design. Although we include these topics in the courses we teach (using this book as the main text), only a few of these applications are broadly enough accessible to be included here.

The aim of part II is to show the reader, by example, how convex optimization can be applied in practice.

1.5.3 Part III: Algorithms

In part III, *Algorithms*, we describe numerical methods for solving convex optimization problems, focusing on Newton's algorithm and interior-point methods. Part III is organized as three chapters, which cover unconstrained optimization, equality constrained optimization, and inequality constrained optimization, respectively. These chapters follow a natural hierarchy, in which solving a problem is reduced to solving a sequence of simpler problems. Quadratic optimization problems (including, *e.g.*, least-squares) form the base of the hierarchy; they can be

solved exactly by solving a set of linear equations. Newton's method, developed in chapters 9 and 10, is the next level in the hierarchy. In Newton's method, solving an unconstrained or equality constrained problem is reduced to solving a sequence of quadratic problems. In chapter 11, we describe interior-point methods, which form the top level of the hierarchy. These methods solve an inequality constrained problem by solving a sequence of unconstrained, or equality constrained, problems.

Overall we cover just a handful of algorithms, and omit entire classes of good methods, such as quasi-Newton, conjugate-gradient, bundle, and cutting-plane algorithms. For the methods we do describe, we give simplified variants, and not the latest, most sophisticated versions. Our choice of algorithms was guided by several criteria. We chose algorithms that are simple (to describe and implement), but also reliable and robust, and effective and fast enough for most problems.

Many users of convex optimization end up using (but not developing) standard software, such as a linear or semidefinite programming solver. For these users, the material in part III is meant to convey the basic flavor of the methods, and give some ideas of their basic attributes. For those few who will end up developing new algorithms, we think that part III serves as a good introduction.

1.5.4 Appendices

There are three appendices. The first lists some basic facts from mathematics that we use, and serves the secondary purpose of setting out our notation. The second appendix covers a fairly particular topic, optimization problems with quadratic objective and one quadratic constraint. These are nonconvex problems that nevertheless can be effectively solved, and we use the results in several of the applications described in part II.

The final appendix gives a brief introduction to numerical linear algebra, concentrating on methods that can exploit problem structure, such as sparsity, to gain efficiency. We do not cover a number of important topics, including roundoff analysis, or give any details of the methods used to carry out the required factorizations. These topics are covered by a number of excellent texts.

1.5.5 Comments on examples

In many places in the text (but particularly in parts II and III, which cover applications and algorithms, respectively) we illustrate ideas using specific examples. In some cases, the examples are chosen (or designed) specifically to illustrate our point; in other cases, the examples are chosen to be 'typical'. This means that the examples were chosen as samples from some obvious or simple probability distribution. The dangers of drawing conclusions about algorithm performance from a few tens or hundreds of randomly generated examples are well known, so we will not repeat them here. These examples are meant only to give a rough idea of algorithm performance, or a rough idea of how the computational effort varies with problem dimensions, and not as accurate predictors of algorithm performance. In particular, your results may vary from ours.

1.5.6 Comments on exercises

Each chapter concludes with a set of exercises. Some involve working out the details of an argument or claim made in the text. Others focus on determining, or establishing, convexity of some given sets, functions, or problems; or more generally, convex optimization problem formulation. Some chapters include numerical exercises, which require some (but not much) programming in an appropriate high level language. The difficulty level of the exercises is mixed, and varies without warning from quite straightforward to rather tricky.

1.6 Notation

Our notation is more or less standard, with a few exceptions. In this section we describe our basic notation; a more complete list appears on page 697.

We use $\mathbf{R}$ to denote the set of real numbers, $\mathbf{R}_+$ to denote the set of nonnegative real numbers, and $\mathbf{R}_{++}$ to denote the set of positive real numbers. The set of real n-vectors is denoted $\mathbf{R}^n$, and the set of real $m \times n$ matrices is denoted $\mathbf{R}^{m\times n}$. We delimit vectors and matrices with square brackets, with the components separated by space. We use parentheses to construct column vectors from comma separated lists. For example, if $a,\ b,\ c \in \mathbf{R}$, we have

$$(a,b,c) = \left[\begin{array}{c} a \\ b \\ c \end{array}\right] = \left[\begin{array}{ccc} a & b & c \end{array}\right]^T,$$

which is an element of $\mathbf{R}^3$. The symbol $\mathbf{1}$ denotes a vector all of whose components are one (with dimension determined from context). The notation x_i can refer to the ith component of the vector x, or to the ith element of a set or sequence of vectors $x_1, x_2, \ldots$. The context, or the text, makes it clear which is meant.

We use $\mathbf{S}^k$ to denote the set of symmetric $k \times k$ matrices, $\mathbf{S}^k_+$ to denote the set of symmetric positive semidefinite $k \times k$ matrices, and $\mathbf{S}^k_{++}$ to denote the set of symmetric positive definite $k \times k$ matrices. The curled inequality symbol $\succeq$ (and its strict form $\succ$) is used to denote generalized inequality: between vectors, it represents componentwise inequality; between symmetric matrices, it represents matrix inequality. With a subscript, the symbol $\preceq_K$ (or $\prec_K$) denotes generalized inequality with respect to the cone K (explained in §2.4.1).

Our notation for describing functions deviates a bit from standard notation, but we hope it will cause no confusion. We use the notation $f : \mathbf{R}^p \to \mathbf{R}^q$ to mean that f is an $\mathbf{R}^q$-valued function on some *subset* of $\mathbf{R}^p$, specifically, its *domain*, which we denote $\mathbf{dom}\, f$. We can think of our use of the notation $f : \mathbf{R}^p \to \mathbf{R}^q$ as a declaration of the function *type*, as in a computer language: $f : \mathbf{R}^p \to \mathbf{R}^q$ means that the function f takes as argument a real p-vector, and returns a real q-vector. The set $\mathbf{dom}\, f$, the domain of the function f, specifies the subset of $\mathbf{R}^p$ of points x for which $f(x)$ is defined. As an example, we describe the logarithm function as $\log : \mathbf{R} \to \mathbf{R}$, with $\mathbf{dom}\log = \mathbf{R}_{++}$. The notation $\log : \mathbf{R} \to \mathbf{R}$ means that

the logarithm function accepts and returns a real number; $\mathbf{dom}\log = \mathbf{R}_{++}$ means that the logarithm is defined only for positive numbers.

We use $\mathbf{R}^n$ as a generic finite-dimensional vector space. We will encounter several other finite-dimensional vector spaces, *e.g.*, the space of polynomials of a variable with a given maximum degree, or the space $\mathbf{S}^k$ of symmetric $k \times k$ matrices. By identifying a basis for a vector space, we can always identify it with $\mathbf{R}^n$ (where n is its dimension), and therefore the generic results, stated for the vector space $\mathbf{R}^n$, can be applied. We usually leave it to the reader to translate general results or statements to other vector spaces. For example, any linear function $f : \mathbf{R}^n \to \mathbf{R}$ can be represented in the form $f(x) = c^T x$, where $c \in \mathbf{R}^n$. The corresponding statement for the vector space $\mathbf{S}^k$ can be found by choosing a basis and translating. This results in the statement: any linear function $f : \mathbf{S}^k \to \mathbf{R}$ can be represented in the form $f(X) = \mathbf{tr}(CX)$, where $C \in \mathbf{S}^k$.

Bibliography

Least-squares is a very old subject; see, for example, the treatise written (in Latin) by Gauss in the 1820s, and recently translated by Stewart [Gau95]. More recent work includes the books by Lawson and Hanson [LH95] and Björck [Bjö96]. References on linear programming can be found in chapter 4.

There are many good texts on local methods for nonlinear programming, including Gill, Murray, and Wright [GMW81], Nocedal and Wright [NW99], Luenberger [Lue84], and Bertsekas [Ber99].

Global optimization is covered in the books by Horst and Pardalos [HP94], Pinter [Pin95], and Tuy [Tuy98]. Using convex optimization to find bounds for nonconvex problems is an active research topic, and addressed in the books above on global optimization, the book by Ben-Tal and Nemirovski [BTN01, §4.3], and the survey by Nesterov, Wolkowicz, and Ye [NWY00]. Some notable papers on this subject are Goemans and Williamson [GW95], Nesterov [Nes00, Nes98], Ye [Ye99], and Parrilo [Par03]. Randomized methods are discussed in Motwani and Raghavan [MR95].

Convex analysis, the mathematics of convex sets, functions, and optimization problems, is a well developed subfield of mathematics. Basic references include the books by Rockafellar [Roc70], Hiriart-Urruty and Lemaréchal [HUL93, HUL01], Borwein and Lewis [BL00], and Bertsekas, Nedić, and Ozdaglar [Ber03]. More references on convex analysis can be found in chapters 2–5.

Nesterov and Nemirovski [NN94] were the first to point out that interior-point methods can solve many convex optimization problems; see also the references in chapter 11. The book by Ben-Tal and Nemirovski [BTN01] covers modern convex optimization, interior-point methods, and applications.

Solution methods for convex optimization that we do not cover in this book include subgradient methods [Sho85], bundle methods [HUL93], cutting-plane methods [Kel60, EM75, GLY96], and the ellipsoid method [Sho91, BGT81].

The idea that convex optimization problems are tractable is not new. It has long been recognized that the theory of convex optimization is far more straightforward (and complete) than the theory of general nonlinear optimization. In this context Rockafellar stated, in his 1993 SIAM Review survey paper [Roc93],

> *In fact the great watershed in optimization isn't between linearity and nonlinearity, but convexity and nonconvexity.*

The first formal argument that convex optimization problems are easier to solve than general nonlinear optimization problems was made by Nemirovski and Yudin, in their 1983 book *Problem Complexity and Method Efficiency in Optimization* [NY83]. They showed that the information-based complexity of convex optimization problems is far lower than that of general nonlinear optimization problems. A more recent book on this topic is Vavasis [Vav91].

The low (theoretical) complexity of interior-point methods is integral to modern research in this area. Much of the research focuses on proving that an interior-point (or other) method can solve some class of convex optimization problems with a number of operations that grows no faster than a polynomial of the problem dimensions and $\log(1/\epsilon)$, where $\epsilon > 0$ is the required accuracy. (We will see some simple results like these in chapter 11.) The first comprehensive work on this topic is the book by Nesterov and Nemirovski [NN94]. Other books include Ben-Tal and Nemirovski [BTN01, lecture 5] and Renegar [Ren01]. The polynomial-time complexity of interior-point methods for various convex optimization problems is in marked contrast to the situation for a number of nonconvex optimization problems, for which all known algorithms require, in the worst case, a number of operations that is exponential in the problem dimensions.

Convex optimization has been used in many applications areas, too numerous to cite here. Convex analysis is central in economics and finance, where it is the basis of many results. For example the separating hyperplane theorem, together with a no-arbitrage assumption, is used to deduce the existence of prices and risk-neutral probabilities (see, *e.g.*, Luenberger [Lue95, Lue98] and Ross [Ros99]). Convex optimization, especially our ability to solve semidefinite programs, has recently received particular attention in automatic control theory. Applications of convex optimization in control theory can be found in the books by Boyd and Barratt [BB91], Boyd, El Ghaoui, Feron, and Balakrishnan [BEFB94], Dahleh and Diaz-Bobillo [DDB95], El Ghaoui and Niculescu [EN00], and Dullerud and Paganini [DP00]. A good example of embedded (convex) optimization is model predictive control, an automatic control technique that requires the solution of a (convex) quadratic program at each step. Model predictive control is now widely used in the chemical process control industry; see Morari and Zafirou [MZ89]. Another applications area where convex optimization (and especially, geometric programming) has a long history is electronic circuit design. Research papers on this topic include Fishburn and Dunlop [FD85], Sapatnekar, Rao, Vaidya, and Kang [SRVK93], and Hershenson, Boyd, and Lee [HBL01]. Luo [Luo03] gives a survey of applications in signal processing and communications. More references on applications of convex optimization can be found in chapters 4 and 6–8.

High quality implementations of recent interior-point methods for convex optimization problems are available in the LOQO [Van97] and MOSEK [MOS02] software packages, and the codes listed in chapter 11. Software systems for specifying optimization problems include AMPL [FGK99] and GAMS [BKMR98]. Both provide some support for recognizing problems that can be transformed to linear programs.

Part I

Theory